

Auditoría de código

Dona — Asistente de IA por WhatsApp

Informe técnico completo

Proyecto	Dona — agente conversacional WhatsApp para PYMES en EEUU
Repositorio	C:\Users\celes\Dona-agent (no es repo git local)
Fecha del informe	2026-04-27
Stack	Python 3.11+ · FastAPI · SQLAlchemy 2 async · PostgreSQL/SQLite
LLMs	Claude Sonnet 4.6 (primario) · DeepSeek · GPT-4o · Haiku (fallback)
Tamaño	~24 700 LOC en agent/ · ~2 000 LOC en enhanced/ · 30 archivos de tests
Tests	411 tests, 100% en verde (pytest, ~34 s)
Estado	Producción con primeros usuarios

Documento generado automáticamente a partir de la lectura completa del código fuente, los archivos de configuración y la suite de pruebas. No incluye datos de producción ni capturas de la base de datos en vivo.

1. Resumen ejecutivo

Dona es un servicio en producción que atiende usuarios reales por WhatsApp combinando un modelo de lenguaje grande con un sistema persistente de memoria, billing por créditos, generación de imagen/video, y un conjunto de integraciones (Gmail, Google Calendar, Sheets, Drive, Stripe, Cloudflare R2). El código presenta una **madurez notable** para un proyecto de un solo desarrollador: separación de capas, fallbacks degradables, sanitización contra prompt injection, billing idempotente y suite de pruebas amplia.

Veredicto general

El sistema es **apto para producción a la escala actual** (primeros usuarios). Sus controles críticos — verificación HMAC de webhooks, idempotencia de Stripe, cifrado de tokens OAuth, rate limiting persistente, sanitización anti-injection — están implementados correctamente y cubiertos por tests. Los riesgos principales son de **escalabilidad** (módulos monolíticos de 2 000–3 000 LOC en brain.py, memory.py, main.py) y de **oscurecimiento operativo** (escasa observabilidad estructurada, ausencia de migraciones Alembic activas pese a estar en requirements).

Hallazgos prioritarios

Sev	Hallazgo	Ubicación	Acción recomendada
ALTA	Cumplimiento CCPA/CPRA todavía manual en /privacy/export y /privacy/delete	agent/main.py:204–257	Implementar flujo OTP por WhatsApp y export JSON automático en ≤45 días.
MEDIA	META_APP_SECRET ausente acepta webhooks sin firma con sólo un warning	agent/providers/meta.py:131–136	Bloquear webhooks no firmados en producción (env != development).
MEDIA	STRIPE_WEBHOOK_SECRET ausente cae a aceptar payloads sin verificar	agent/billing.py:371–377	Fallar cerrado en producción; sólo permitir bypass en tests.
MEDIA	brain.py / memory.py / main.py exceden 2 000 LOC cada uno	agent/brain.py (3 319), agent/memory.py (2 316), agent/main.py (2 170)	Refactor incremental: extraer routers FastAPI, capa de modelos, dispatcher de tools.
MEDIA	Alembic en requirements.txt pero no se aplican migraciones	agent/memory.py:Base.metadata.create_all (lifespan)	Inicializar repositorio de migraciones antes de añadir más tablas en producción.
BAJA	Backend de jobs `inproc` no persiste tras restart	agent/jobs/queue.py:196–208	Documentar como `dev-only`; forzar arq+Redis en producción vía health-check.
BAJA	Hash sin salt del teléfono en la key de R2	agent/storage.py:65–70	Aceptable (no es secreto); agregar nota en docs y considerar HMAC con secret.
BAJA	Comentarios `AgentKit` legacy en partes del código	Comentarios dispersos (declarado en CLAUDE.md)	Limpieza cosmética; no bloquea.
INFO	411 tests pasan en ~34 s; sin medición de cobertura	tests/	Activar `pytest --cov=agent` y subir umbral progresivamente (objetivo ≥75%).

2. Arquitectura y estructura

El árbol del proyecto separa claramente las responsabilidades. **agent/** concentra toda la lógica del producto; **enhanced/** es un módulo más reciente para sistemas diagnósticos y procesadores especializados; **tests/** espeja parcialmente la estructura de agent/.

2.1 Mapa de módulos por LOC

agent/brain.py — 3 319 LOC	Cerebro: cliente Anthropic, fallbacks (DeepSeek, GPT-4o, Haiku), selección dinámica de t
agent/memory.py — 2 316 LOC	26 modelos SQLAlchemy, sesiones async, helpers de persistencia para mensajes, record
agent/main.py — 2 170 LOC	FastAPI app, middleware (logging+security headers), webhooks Whapi/Meta/Stripe, endp
agent/creativos/comandos.py — 992 LOC	Detección y dispatch de comandos creativos (imagen, video, voz, PDF, ajustes).
agent/jobs/handlers_creativos.py — 758 LOC	Lógica de ejecución de jobs creativos (integraciones Gemini, Replicate, HeyGen).
agent/proactivity.py — 746 LOC	Mensajería proactiva: resumen matutino, recordatorios, follow-ups.
agent/providers/meta.py — 600 LOC	Adaptador Meta WhatsApp Cloud API (HMAC + replay protection).
agent/scheduler.py — 577 LOC	APScheduler para envíos programados y proactivos.
enhanced/* — ~1 990 LOC	Catálogo de sistemas, ejecución segura (safe_module), monitoreo, NLP detector.

2.2 Capas y flujo principal

El flujo de un mensaje entrante atraviesa estas capas en orden:

- **Webhook FastAPI** (main.py) recibe POST de Whapi/Meta y verifica firma.
- **Deduplicación** por mensaje_id contra Postgres con fallback in-memory.
- **Rate limiting** sliding-window (Redis o memoria) — 10/min usuario, 1 000/min global.
- **Detección determinística** de comandos en main.py: STOP/START, saldo, recargar, imagen, privacidad, onboarding. Estos NO consumen LLM ni créditos.
- **Brain** (brain.py) clasifica intent, selecciona tools, llama a Claude con fallbacks.
- **Tools de 2 pasos:** preparar_X → confirmar_X / cancelar_X (literal en el nombre).
- **Persistencia** de mensajes y memoria larga via SQLAlchemy async.
- **Respuesta** via proveedor de WhatsApp (Whapi default; Meta/Twilio según config).
- **Jobs creativos** (imagen/video) van a cola arq (Redis) o inproc (asyncio.create_task).

2.3 Convenciones documentadas en CLAUDE.md

El archivo CLAUDE.md codifica decisiones que *no son derivables del código*: español como idioma único, prohibición del término “Fundadores”, marco legal US (no LFPDPPP), patrón preparar/confirmar/cancelar literal para tools con efecto secundario, regex previa al LLM para comandos comunes, integer-only credits con audit trail. Esta documentación viva es uno de los activos más valiosos del repo.

3. Seguridad

3.1 Verificación HMAC de webhooks

Meta (providers/meta.py) verifica X-Hub-Signature-256 con `hmac.compare_digest`; agrega replay protection rechazando mensajes con antigüedad > 600s o timestamp futuro > 300s. **Stripe** (billing.py) usa `stripe.Webhook.construct_event`. **Inbound tokens** (inbound_tokens.py) firman con HMAC-SHA256 con secreto de entorno y verifican timing-safe.

Riesgo: ambos verificadores caen al modo "aceptar sin firma" cuando falta la variable de entorno (`META_APP_SECRET`, `STRIPE_WEBHOOK_SECRET`) y sólo emiten warning. En desarrollo es razonable; en producción debería abortar.

3.2 Cifrado de tokens OAuth

`agent/crypto.py` usa Fernet (AES-128-CBC + HMAC-SHA256). El módulo **aborta el arranque** si `ENVIRONMENT=production` y falta `ENCRYPTION_KEY` (o si la clave es inválida). En dev opera transparente con warning. Esta decisión — *fail closed en prod, fail open en dev* — es la correcta para el modelo de amenaza.

3.3 Defensa contra prompt injection

`brain.py` define `_PATRONES_INYECCION` y `_sanitizar_datos_externos()` que se aplica a TODOS los datos externos antes de inyectarlos como resultado de tool: memoria vectorial, memoria de largo plazo, eventos de calendario (título y lugar), correos (subject, snippet y cuerpo). Patrones cubiertos: ignore/forget/override de instrucciones, "act as / actúa como", marcadores system-prompt, "send a message to", "reveal prompt/api key". El contenido sospechoso se reemplaza por [contenido filtrado por seguridad] y todo bloque externo va envuelto en `<external_data>`.

Cobertura observada: 14 puntos de inyección sanitizan correctamente. Las búsquedas en correo electrónico, eventos de Google Calendar y memoria larga están todas blindadas.

3.4 Rate limiting

`rate_limiter.py` implementa sliding window con sorted sets de Redis (operación atómica via pipeline) y fallback a un dict en memoria con limpieza periódica cuando supera 500 claves. Límites: 10 mensajes/usuario/minuto y 1 000 globales/minuto. El "undo" cuando se excede el límite (líneas 110–119) evita falsos positivos en concurrencia.

3.5 Headers HTTP y middleware

`SecurityHeadersMiddleware` añade en cada respuesta: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `X-XSS-Protection`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` denegando cámara/micrófono/geolocation, y `Cache-Control: no-store` excepto en rutas `/auth/`. `RequestLoggingMiddleware` mide latencia y eleva a WARNING los requests > 5 s.

3.6 Autenticación admin

`_verificar_admin()` exige `Authorization: Bearer <ADMIN_TOKEN>` y compara con `hmac.compare_digest`. Acepta `query param` como `fallback legacy` (menos seguro pero también `timing-safe`). Recomendación: deprecate el `query param` en futuros endpoints nuevos.

3.7 Búsqueda de antipatrones

Búsqueda exhaustiva en `agent/` de patrones inseguros:

- Sin uso de `eval()`, `exec()`, `os.system()` ni `subprocess`.
- Sin `shell=True`; sin `pickle.load`.
- Sin `except: bare` ni `except Exception: pass`.
- Sin claves API hardcodeadas (`sk-*`, `AKIA*`).
- `md5` usado sólo como hash de URL para deduplicación de noticias (`real_world.py:146`) — uso no criptográfico, aceptable.
- `sha1` usado sólo como ofuscador de teléfono en keys de R2 (`storage.py:70`) — declarado explícitamente como `no-secreto`.

4. Cumplimiento legal

El usuario opera desde EEUU; el marco legal aplicable, según CLAUDE.md, es **CCPA/CPRA + FTC + TCPA**, no marcos mexicanos. La implementación actual cubre los pilares pero deja trabajo manual en derechos de acceso/borrado.

4.1 TCPA — opt-in / opt-out

Comandos STOP y START (case-insensitive, con o sin “dona”) se evalúan *antes* de cualquier otro routing y no consumen LLM ni créditos. Esta es la ruta correcta: TCPA exige acción inmediata. Se recomienda añadir tests de regresión que verifiquen que ningún cambio futuro mueva ese check después del LLM.

4.2 CCPA / CPRA — derechos del consumidor

Páginas legales: /privacy y /terms servidos desde agent/legal_pages.py (HTML estático). **Endpoints CCPA:** /privacy/export y /privacy/delete existen pero hoy retornan *instrucciones de contacto* en vez de procesar la solicitud automáticamente. Esto es legal — CCPA permite verificar identidad y responder en hasta 45 días — pero **requiere infraestructura humana** para cumplir esa ventana.

Riesgo medio: a medida que crece la base de usuarios, el flujo manual no escala. Implementar OTP por WhatsApp + export JSON automático evita SLA misses regulatorios.

4.3 Subprocessors

legal_pages.py declara explícitamente los subprocesadores en la política (Anthropic, OpenAI/DeepSeek, Google, Stripe, Cloudflare, Render, etc.), lo cual es buena práctica CCPA y exigible bajo SCC para usuarios EU si aplica.

4.4 Datos sensibles y minimización

- El número de teléfono se almacena en claro (clave primaria); las keys de R2 lo ofuscan con SHA-1 truncado.
- Los tokens OAuth se cifran con Fernet (cuando ENCRYPTION_KEY está configurado).
- El historial de mensajes se persiste sin redacción — necesario para contexto del LLM, pero implica que un export CCPA debe poder devolverlo y un delete debe borrarlo todo.
- memory.py tiene una función documentada (línea 1984) para borrar TODO de un usuario en todas las tablas — base correcta para implementar delete automático.

5. Billing — créditos, Stripe e idempotencia

agent/billing.py implementa el sistema de créditos con tres operaciones atómicas y un webhook idempotente.

5.1 Cobro atómico (compare-and-swap)

cobrar() evita SELECT FOR UPDATE (no portable entre SQLite y Postgres) usando un UPDATE condicional WHERE saldo >= credits y verificando rowcount. Si dos requests intentan cobrar al mismo tiempo y sólo alcanza para una, la otra recibe rowcount=0 y obtiene SaldoInsuficienteError. Cada cobro deja una fila en transacciones_credito con saldo resultante (audit trail completo).

5.2 Acreditación idempotente

acreditar() consulta primero stripe_session_id en transacciones_credito; si existe, retorna el saldo actual sin hacer nada. Esto resuelve correctamente el contrato "at-least-once" de los webhooks de Stripe — una reentrega del mismo evento no duplica créditos. La idempotencia se valida con tests (tests/test_billing.py).

Bien resuelto: el patrón captura saldo_prev = int(row.saldo) antes del UPDATE para evitar la auto-refresh del ORM (comentario explícito en línea 264).

5.3 Helper preparar/cobrar

cobrar_o_rechazar() es el helper que toda tool pagada debe usar antes de trabajo caro. Devuelve (False, mensaje_para_usuario) con texto listo para WhatsApp + CTA para recargar. Esto centraliza UX de saldo bajo.

5.4 Stripe Checkout

crear_checkout() envuelve la llamada sync del SDK de Stripe en asyncio.to_thread para no bloquear el event loop, lo que es correcto. El client_reference_id y metadata portan el teléfono y créditos para que el webhook pueda acreditar sin sesiones internas.

5.5 Webhook Stripe — verificación

verificar_firma_stripe() usa stripe.Webhook.construct_event cuando hay secreto. **Cuando NO hay secreto, parsea el JSON sin verificar y emite un warning.** Aceptable en dev; en producción debería rechazar (HTTP 400) en vez de aceptar payloads no firmados.

6. Sistema de jobs y storage

6.1 Cola de jobs (arq + inproc)

agent/jobs/queue.py abstrae dos backends detrás de la misma API:

- **arq** (Redis): persistente, distribuible, recomendado para producción.
- **inproc** (asyncio.create_task): no persiste tras restart, sólo dev.
- Auto-detección: si REDIS_URL está seteada → arq; sino → inproc.
- Cada job persiste como JobCreativo en DB con estado pending → running → done/error y contador de intentos.
- Si arq.enqueue_job() falla, hay fallback automático a inproc dentro del mismo request.

Recomendación: agregar un health-check que falle si ENVIRONMENT=production y el backend efectivo es inproc — para evitar que un mal config llegue a producción sin Redis y se pierdan jobs en cada redeploy.

6.2 Storage de assets (R2 + filesystem)

agent/storage.py usa Cloudflare R2 (S3-compatible vía aioboto3, sin egress fees) como primario, con fallback a filesystem local. Las keys siguen el esquema {hash_telefono}/{tipo}/{uuid}.{ext}; el tipo se sanitiza a [a-z0-9_-] para evitar inyecciones de path. Cada subida queda registrada en asset_generado para audit y billing.

6.3 Modelo de datos (26 tablas)

memory.py declara 26 modelos SQLAlchemy. Categorías:

- **Conversación:** Mensaje, MensajeProcesado (dedup), SesionConversacion, MemoriaLargoPlazo.
- **Usuario:** TimezoneUsuario, UsuarioUbicacion, UsuarioOnboarding, UsuarioProactividad, UsuarioGoogleAuth, UsuarioMiroFish, PerfilAprendizaje, UsuarioEstadoEmocional.
- **Tareas y agenda:** Recordatorio, RecordatorioGCalEnviado, Tarea, Lista, ItemLista, EventoUsuario.
- **Aprendizaje y eventos:** EventoEmocional, EventoComportamiento, NoticiaEnviada.
- **Billing y assets:** SaldoCreditos, TransaccionCredito, AssetGenerado, JobCreativo.

Riesgo medio: el esquema crece via Base.metadata.create_all en lifespan, no via Alembic. Funciona para desarrollo y para deployments greenfield, pero **cualquier cambio breaking** (rename de columna, cambio de tipo, índice nuevo) en un schema con datos requiere migración. Alembic está en requirements.txt pero no veo alembic.ini ni carpeta versions/. Iniciar el repositorio de migraciones **antes** del próximo cambio de schema.

7. Observabilidad

7.1 Logging

Logger único (`logging.getLogger("dona")`) configurado en `agent/logging_config.py`. Convención de prefijos por subsistema: [BRAIN], [BILLING], [META], [STRIPE], [RATE], [CRYPTO], [SECURITY], [JOBS], [INBOUND]. Esto facilita filtrado posterior aunque no haya logging estructurado JSON.

7.2 Métricas in-memory

La clase `_Metricas` en `main.py` expone vía `/admin/metrics`: requests totales, requests por ruta, errores 5xx, mensajes procesados, latencia promedio y count de requests > 5 s. Está protegido por Bearer token. **Limitación:** contadores no se persisten ni se exportan a Prometheus, y se reinician en cada redeploy.

7.3 Endpoints admin

- GET `/admin/metrics` — contadores in-memory.
- GET `/admin/jobs-recientes?telefono=...&limite=10` — debug de jobs creativos.
- POST `/admin/seed-creditos` — acreditación manual.
- GET `/admin/recordatorios?telefono=...` — recordatorios programados.
- GET `/admin/onboarding?telefono=...` y POST `/admin/onboarding/reset`.
- GET `/admin/inbound/token?telefono=...` — generar token Zapier/Make/n8n.
- GET `/admin/r2-check` — health-check de R2.

Recomendación: añadir `/admin/health` que verifique en una sola llamada DB, Redis, R2, Anthropic API, Stripe, y devuelva un cuadro consolidado.

8. Testing y deuda técnica

8.1 Cobertura por archivo

30 archivos de tests, 411 tests, ~4 600 LOC, todos en verde en ~34 s. Los archivos se alinean con los módulos críticos:

- **Billing:** `test_billing.py` + `test_billing_commands.py` + `test_quotas.py`.
- **Creativos:** `test_creativos_imagen.py`, `test_creativos_ajustar.py`, `test_creativos_prompt_imagen.py`.
- **Seguridad:** `test_security_fixes.py`, `test_inbound_tokens.py`.
- **Providers:** `test_providers.py`.
- **Jobs:** `test_jobs.py`.
- **Storage:** `test_storage.py`.
- **Brain:** `test_brain_tools.py`.
- **Onboarding y business:** `test_onboarding_negocio.py`, `test_business.py`, `test_reminders_nl.py`.

- **Integraciones Google:** test_gmail_*.py, test_google_*.py.
- **End-to-end:** test_smoke_e2e.py.

8.2 Patrones de fixtures

conftest.py setea ENVIRONMENT=test, DATABASE_URL=sqlite+aiosqlite:///./test_dona.db y una ANTHROPIC_API_KEY ficticia. Los tests usan SQLite temporal con fixtures que recargan módulos via importlib.reload tras setear env vars — esto permite probar módulos que cachean estado en import-time (e.g., crypto.py, billing.py). Mocks de httpx evitan llamadas reales a Gemini/OpenAI/Whapi/Stripe.

8.3 Brechas de cobertura observadas

- No hay pytest --cov en CI; la cobertura porcentual es desconocida.
- brain.py es el archivo más grande pero también el más difícil de testear (LLM en el path crítico). Los tests existentes mockean Claude pero podrían cubrir más ramas del dispatch de tools.
- scheduler.py y proactivity.py tienen lógica temporal compleja; un test de regresión con freeze-time sería valioso.
- main.py webhooks aceptan flujos largos — los tests cubren los caminos felices, pero no veo tests de rechazo de firma Meta inválida ni de Stripe.

8.4 Deuda técnica clasificada

Sev	Hallazgo	Ubicación	Acción recomendada
MEDIA	Monolitos: brain.py 3 319 LOC, memory.py 2 316 LOC, main.py 2 170 LOC	Múltiple	Refactor incremental: extraer routers, capa de modelos por dominio, dispatcher de tools.
MEDIA	Sin migraciones Alembic activas pese a estar en deps	agent/memory.py	Inicializar repo de migraciones antes del próximo cambio de schema.
BAJA	Comentarios legacy de AgentKit (declarado en CLAUDE.md)	Disperso	Limpieza cosmética cuando se modifique el módulo.
BAJA	Hash teléfono SHA-1 sin salt en R2 keys	agent/storage.py:65	Migrar a HMAC con secret rotatable; aceptable hoy.
BAJA	Métricas no persistentes ni exportables	agent/main.py:_Metricas	Exponer endpoint Prometheus o enviar a sink externo.
INFO	Documentación viva concentrada en CLAUDE.md y comentarios	Repo	Mantener — es uno de los mayores activos.

9. Recomendaciones priorizadas

9.1 Inmediatas (próximas 2 semanas)

- **Endurecer producción:** en `verificar_firma_stripe` y `_verificar_firma` de Meta, fallar cerrado cuando `ENVIRONMENT=production` y el secreto falte.
- **Cobertura:** activar `pytest --cov=agent --cov-report=term-missing` y fijar un piso (e.g., 65%) en pre-commit.
- **Tests negativos de webhook:** añadir 4 tests que verifiquen rechazo con firma inválida (Meta), payload sin firma (Stripe), timestamp viejo (Meta) y token HMAC manipulado (inbound).
- **Migraciones:** ejecutar `alembic init` y stampar el schema actual antes del próximo cambio de columna en `memory.py`.

9.2 Corto plazo (próximo mes)

- **CCPA automático:** implementar OTP por WhatsApp para `/privacy/export` (retorna ZIP con JSON de todas las tablas) y `/privacy/delete` (ejecuta el helper de borrado de `memory.py:1984`).
- **Health-check producción:** `/admin/health` que verifique DB, Redis, R2, Anthropic, Stripe y devuelva matriz consolidada. Bloquear deploy si `JOBS_BACKEND=inproc` en prod.
- **Observabilidad estructurada:** migrar logger a JSON (e.g., `python-json-logger`) y exportar las métricas in-memory a Prometheus o un sink HTTP.

9.3 Mediano plazo (próximos 3 meses)

- **Refactor `brain.py`:** extraer `_PATRONES_INYECCION` y `_sanitizar_datos_externos` a `agent/security/sanitize.py`; extraer cada categoría de tool a su propio módulo en `agent/tools/`.
- **Refactor `memory.py`:** dividir en sub-paquetes por dominio (`memory/billing.py`, `memory/conversacion.py`, `memory/onboarding.py`, etc.). Reduce blast-radius de cada cambio.
- **Refactor `main.py`:** mover endpoints a routers FastAPI por dominio (`routers/admin.py`, `routers/legal.py`, `routers/webhooks.py`).
- **Cifrado at-rest:** evaluar cifrar campos sensibles de mensajes (PII) además de tokens OAuth.

9.4 Largo plazo / aspiracional

- **Multi-tenant safety:** aunque CLAUDE.md afirma que NO es multi-tenant, todo el código ya está particionado por telefono. Documentar formalmente el modelo de aislamiento.
- **Pipeline de evaluación:** dataset de mensajes reales redactados + suite que mida calidad de respuesta del LLM en cada deploy.
- **Retention policy:** establecer cuánto tiempo se guarda el historial de mensajes y aplicar borrado automático (relevante para CCPA y para cost-control).

10. Conclusión

Dona es un producto técnicamente **bien construido para su etapa**. La calidad del código supera lo habitual en proyectos de un solo desarrollador: separación de capas, documentación viva, fallbacks en cada integración externa, controles de seguridad implementados de verdad (no de fachada), suite de pruebas robusta y patrones explícitos para evitar que el LLM alucine éxito en operaciones irreversibles.

Los riesgos identificados son **de operación y escala**, no de diseño: endurecer la configuración de producción (fail-closed en webhooks), iniciar el repositorio de migraciones, automatizar los flujos CCPA y refactorizar los tres archivos monolito antes de que sumen más LOC. Ninguno de estos hallazgos bloquea el servicio actual; todos son trabajos manejables incrementalmente.

Si tuviera que priorizar una sola acción: cerrar el flujo CCPA automático con OTP por WhatsApp. Es la única deuda con un reloj regulatorio de 45 días.

— Fin del informe —